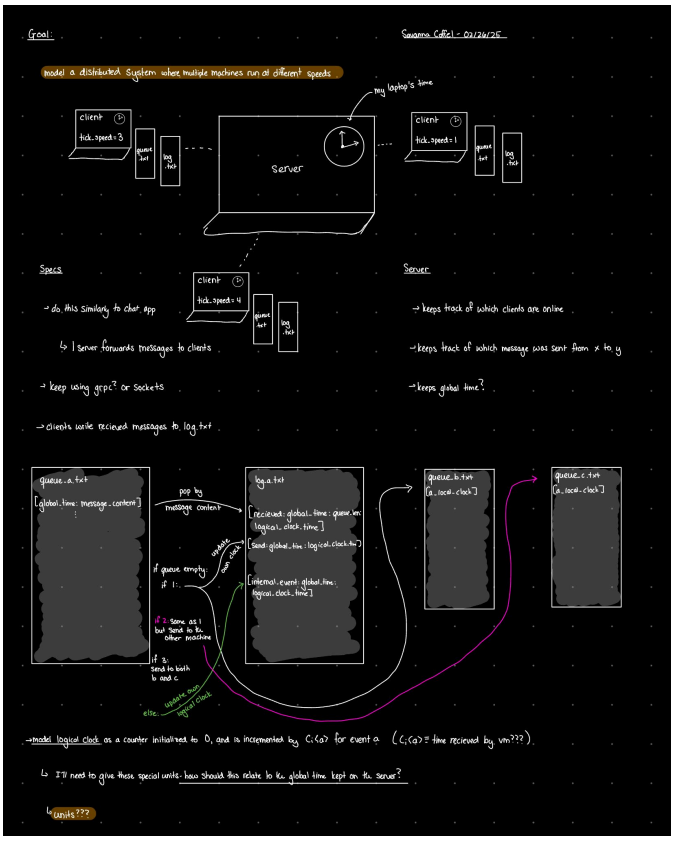
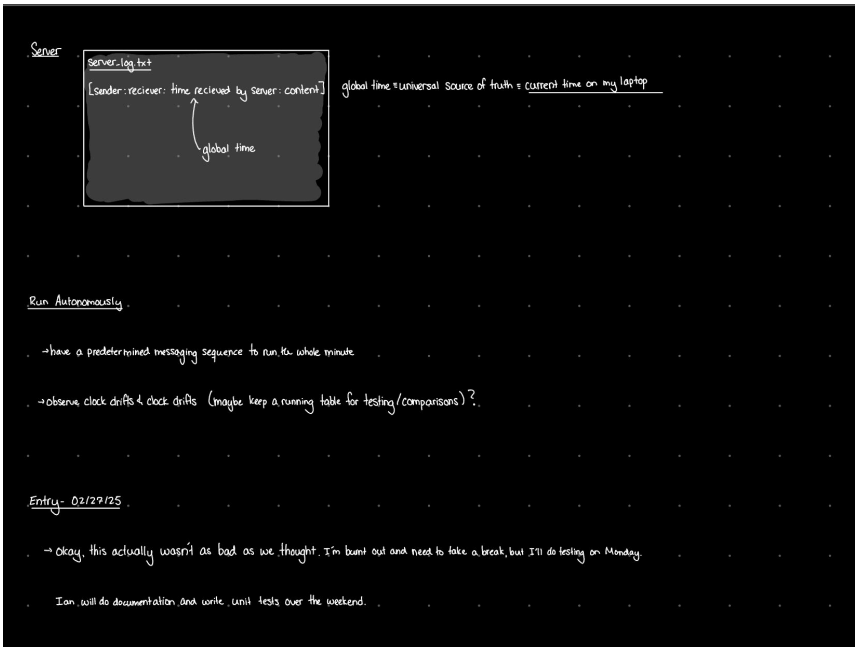
Engineering Notebook - Savanna Coffel and Ian Moore: DE2

Design Specs (Canvas)

I designed this similarly to Design Exercise 1. Now, I can simulate three clients and have a server pass messages between them by forwarding messages, and reuse much of the same design from before:



A notable difference is that now, each “client” has its own logical clock to update, and the server keeps a global time. I do not consider the server a virtual machine, only the clients.



Every client is a separate process so that we can manage multiple processes and preserve process isolation. The main driver code will simulate the clients talking to each other, and block until that iteration is finished. Then, it will continue with another simulation and fresh processes.

Each client process runs concurrent threads for sending/receiving messages, and for pulling from the queue. These threads are self-contained to the client process, so we can still enforce process isolation and thread safety here. It also makes sense from a design standpoint, I do want to be able to process the queue and send messages simultaneously.

Testing Results (Data from 3 March 2025)

Setup:

- Preloaded a couple of messages in the queue to be randomly sent off through the simulation, ensuring a nonempty queue.
- Continued incrementing the machine clock time to track incoming events relative to the machine.

Trial Outcomes:

Trial 1:

- Clock rate of a: 1 tick/s
- Clock rate of b: 3 ticks/s
- Clock rate of c: 6 ticks/s

Trial 2:

- Clock rate of a: 1 tick/s
- Clock rate of b: 5 ticks/s
- Clock rate of c: 2 ticks/s

Trial 3:

- Clock rate of a: 1 tick/s
- Clock rate of b: 6 ticks/s
- Clock rate of c: 2 ticks/s

Trial 4:

- Clock rate of a: 3 ticks/s

- Clock rate of b: 5 ticks/s
- Clock rate of c: 6 ticks/s

Trial 5:

- Clock rate of a: 4 ticks/s
- Clock rate of b: 1 tick/s
- Clock rate of c: 5 ticks/s

Observations and Conclusions:

- **Size of Jumps in Values for Logical Clocks:** Towards the start of each simulation, we see increments of 1, but as the simulation continues, the clocks start to jump by 2s occasionally, and towards the end of the simulation, we even see some jumps in 3s.
- **Drift of the Clocks:** These clocks have closer rates as compared to some of the other trials. We'll definitely see clock drift, especially when grabbing messages from the queue repeatedly. The longer the simulation runs, and the more drastic the differences between the clock rates, the more clock drift we're likely to see.
- **Impact of Different Timings on Gaps in the Clock Values and Length of Message Queue:** When we pull messages from the queue repeatedly, the logical clocks get updated relatively linearly, so we're not very likely to see a clock gap here. If the message queue is empty, or if we're alternating between a length of 0 and a length of 1, we might see some variance for some of the incoming messages. Again, we won't see as drastic of a gap since the clock rates only differ by 1.

Described Variations and Some Bonus Findings: Clock Cycle Variations, Higher clock rates, and longer duration of simulations:

(Sidenote post-demo day: I really want to incorporate more command-line arguments into my simulations. When I did variance testing, I was changing variables in-code, such as the sim duration and the range of possible clock rates. It would be easier and a lot cooler to do this from the command line. A note for next time!)

- If we change the probability of internal event generation by only having an internal event generated for 3/10 numbers, it seems like the clocks tend to drift more. This would make sense because of the time discrepancy between reading messages from the queue and receiving incoming messages, both of which would increment the logical clock compared to a single internal event. So this is not unbelievable.

- If we have smaller variations in the clock cycles, the clocks seem to drift less. This also makes sense, because there are less repeated incrementations of the logical clocks when events are relatively synchronized.
- If you tick up the clock rate, you'll see a lot more variance with the amount of clock drift, especially if the simulation runs for a longer period of time. We have some of these tests in our log files, which are separated by simulation run, so we can compare points of data. It does seem like the overall behavior is the same, but since the variance between clocks are so much higher, we'll see much more drift.
- Finally, for funzies, we let the simulation run just once, for 10 minutes to see what would happen, with the normal probability of internal events at 7/10, and clock rates between 1-6. We again saw an exaggerated version of the behavior from earlier, with more amounts of clock drift because the simulation ran for so long.

Notes on Testing:

We worked very hard to achieve a high level of test coverage to ensure the quality of our codebase. We wrote comprehensive unit tests for both the client and server. Our test cases are numerous and spread over several different files as we worked to increase coverage.

Our test coverage for the client covers unit tests for functions that get called. We leave the driver code alone, since that is dependent on command line arguments and wouldn't be suitable for a unit test suite. We have a detailed breakdown of the untested lines and why they are difficult to test in `README.md`

We have near full test coverage for the server, where the details are in our `README.md`. See this for more information about our client and server tests.

Overall we find that after diligent work on our test suite we are very satisfied with our efforts to ensure quality control.